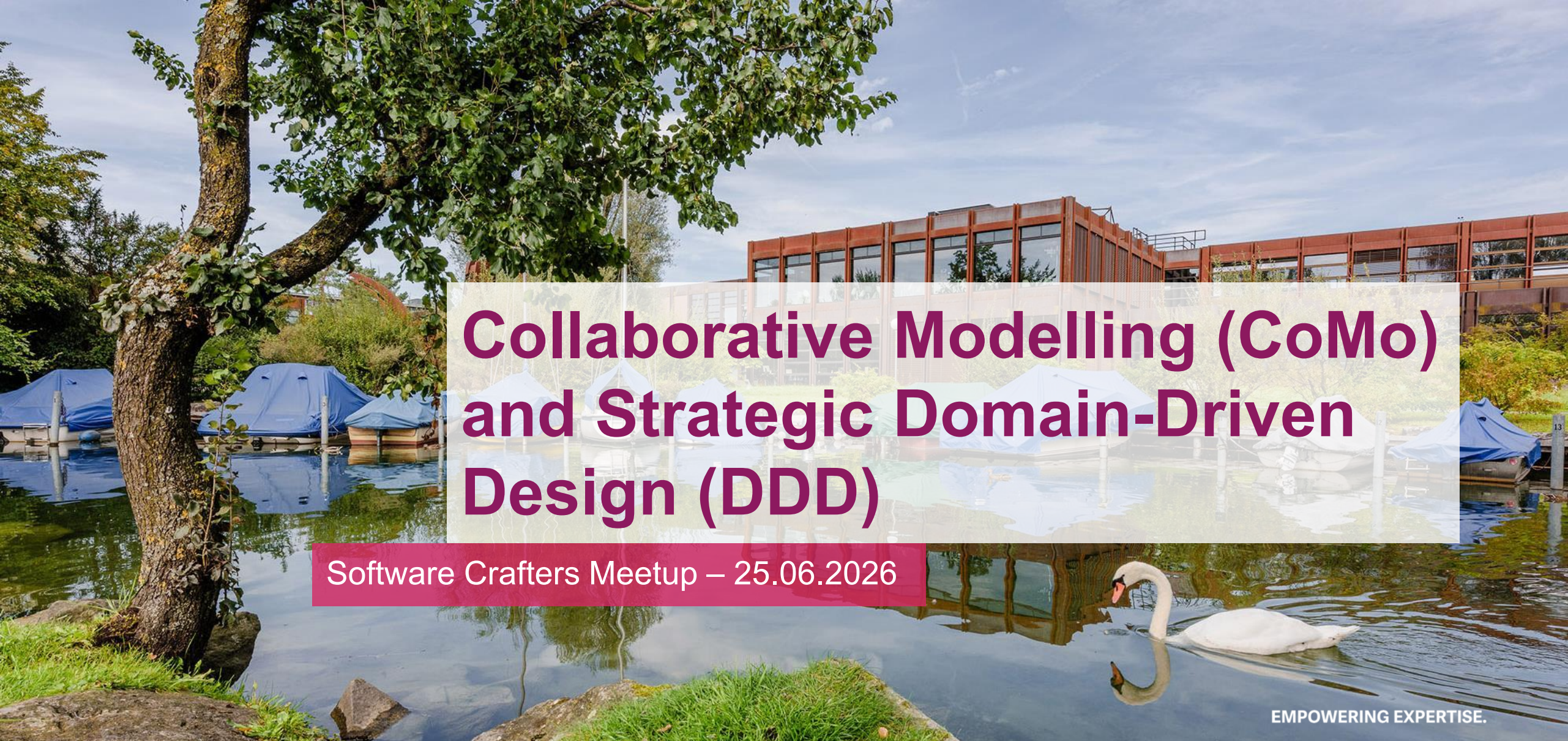


A scenic background image featuring a calm lake. On the left, a large, leafy tree stands on a grassy bank. In the middle ground, several small boats are docked, covered with blue tarps. In the background, a modern building with a prominent red-brown metal frame and large glass windows is visible. A white swan is swimming in the water in the lower right foreground. The sky is overcast with soft clouds.

Collaborative Modelling (CoMo) and Strategic Domain-Driven Design (DDD)

Software Crafters Meetup – 25.06.2026

EMPOWERING EXPERTISE.

Agenda Today

- Short Introduction to CoMo (Motivation)
 - Domain Storytelling (DST)
 - Event Storming
- Practice: EventStorming (Case Study)
- Bridge to Strategic Domain-Driven Design (DDD)
 - Context Mapping and Strategic Patterns
- Outlook

Why Collaborative Modelling (CoMo)?

Motivation

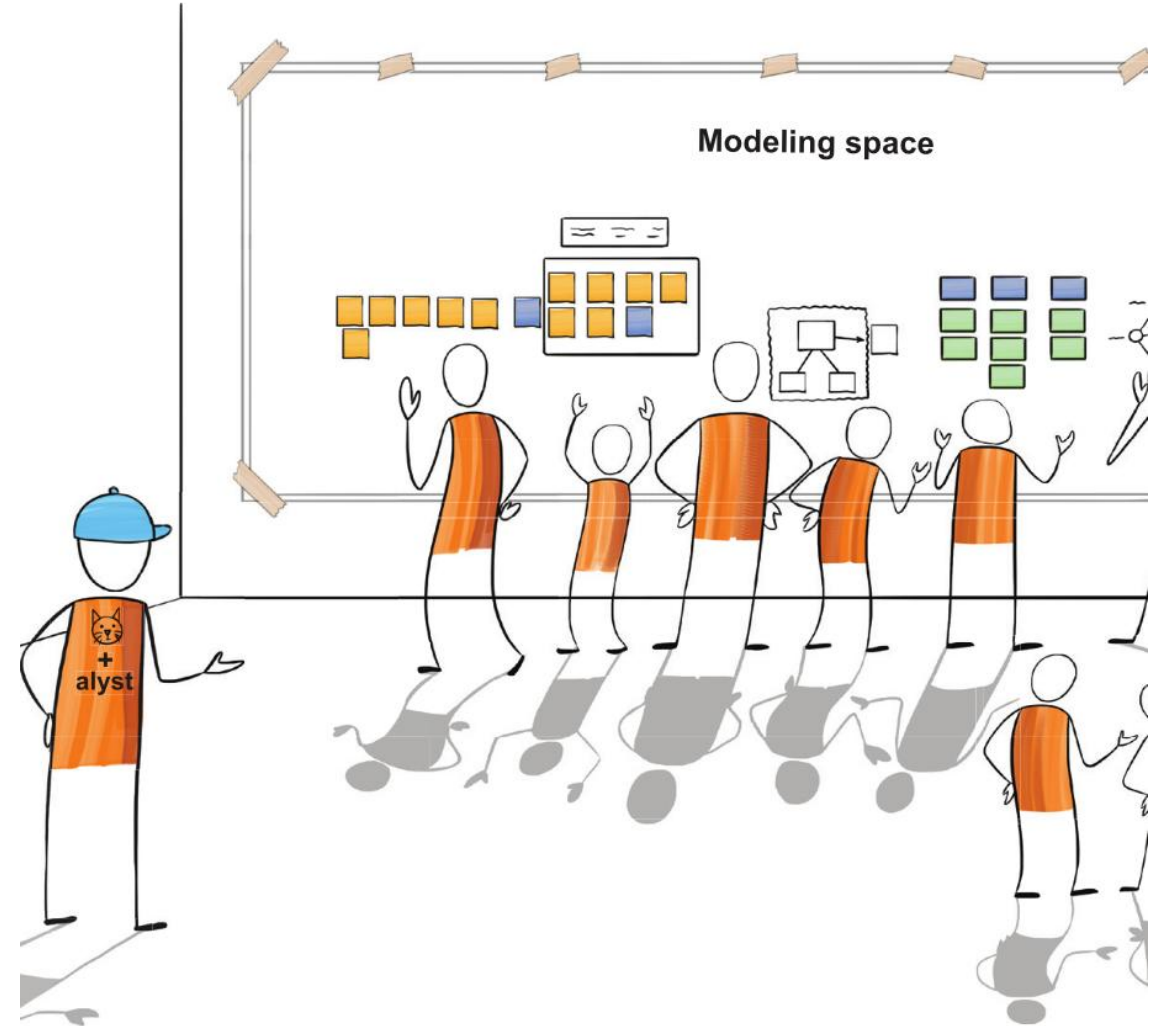


Image Reference: [Collaborative Software Design](#), Evelyn van Kelle, Gien Verschatse, Kenny Baas-Schwegler

Motivation for Domain Modelling

- Capture the **essence** of the **problem domain**
- Establish a **shared understanding** (“ubiquitous language”)
- Align **mental models** through **visualization**
- Align **language** with **business** (domain experts)
 - Bridging technical and non-technical stakeholders
- And later of course also to:
 - Find the right **abstractions** and **simplifications**
 - Use domain knowledge as basis for **system design** and **architecture**

Avoid Misunderstandings ...



*„There are only two hard things in
Computer Science: cache
invalidation and naming things.“*

-- Phil Karlton

Image Reference: Michael Plöd, *Aligning organization and architecture with strategic DDD*, [Link to Slides](#)

Align Solution and „Real World“

Business language	vs.	Developer language
Domain Know-How	vs.	Software Engineering Know-How
Problem space	vs.	Solution space (software design)
Object-oriented analysis (OOA)	vs.	Object-oriented design (OOD)

Subdomains

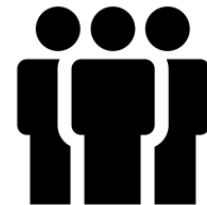
vs.

Bounded Contexts



Business Domain
Experts

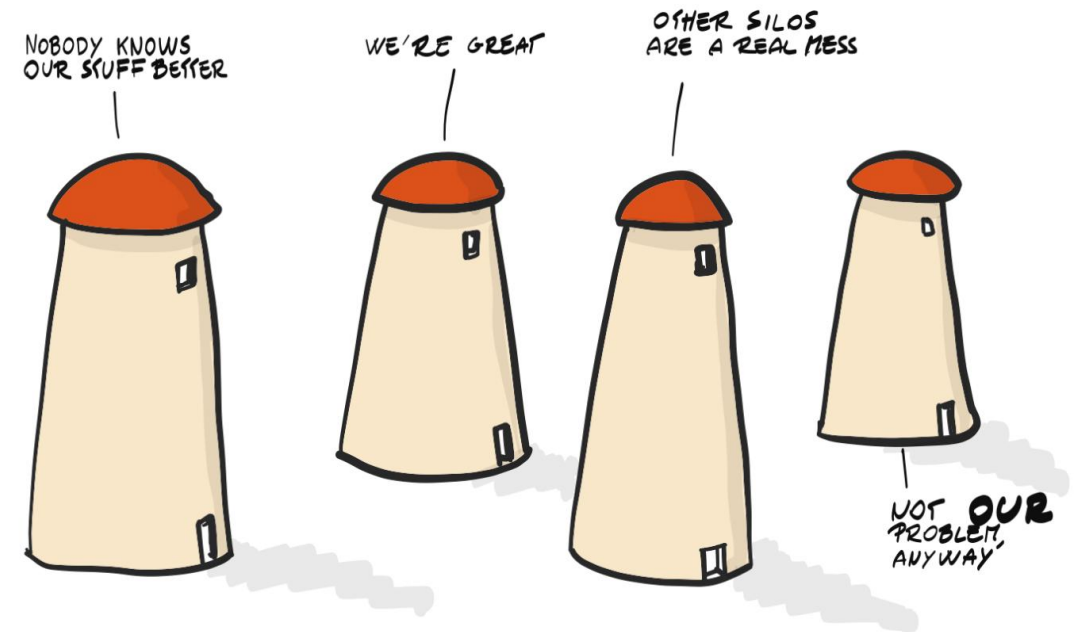
communication required!



Software Engineers
and Architects
("Techies")

Stakeholder Involvement is Key

- Projects within complex domains need **shared understanding**
- Knowledge is typically **spread** ...
 - ... across **business experts, developers, designers, operations, management**, etc.
 - ... and **silos** in organisations
- Without **everyone's perspective**, important goals, rules, expectations and exceptions might be overlooked
- Models must serve as **communication tools**, not just documentation!



Everyone is a master in their silo

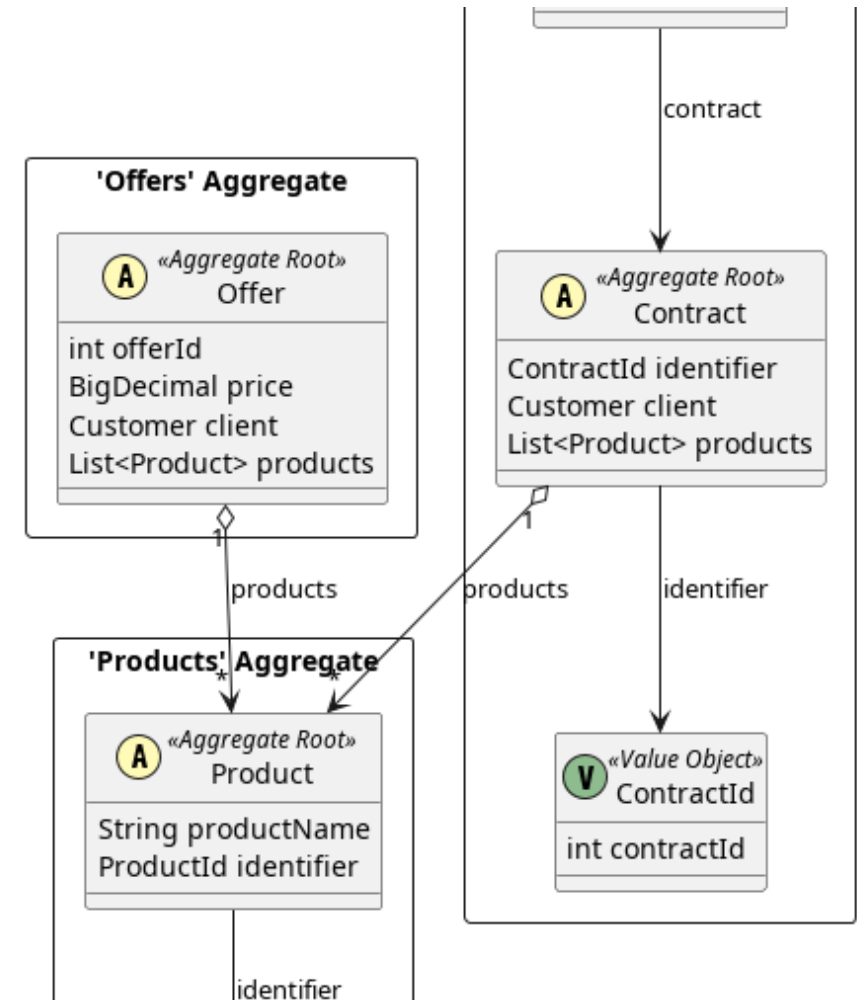
Image Reference: Introducing EventStorming, Alberto Brandolini, http://leanpub.com/introducing_eventstorming

Challenges of Traditional Modelling

Classical notations (such as UML, BPMN, etc.) are designed for **precision** and **technical clarity**...

But they...

- are **not accessible** to non-technical stakeholders (business and domain experts).
- create „**ivory tower**“ **diagrams**, only understood by analysts and „techies“.
- **do not help having the right discussion.**
 - “In fact, using UML severely harms the possibility of this discussion to happen” (Brandolini)
 - They often force discussions into arguing about technical details.
 - CoMo practices support the idea of having an “incremental notation”.



Key Ideas of „Collaborative Modelling“ (CoMo)

- **Pragmatic, Workshop-based, techniques to model collaboratively**
- Invite all team members, important stakeholders and domain experts
- No knowledge on notation required
- Ideally not much preparation by participants required
 - But by moderator/facilitator!
- **Benefits:** inclusion, engagement, speed, shared language, discovery

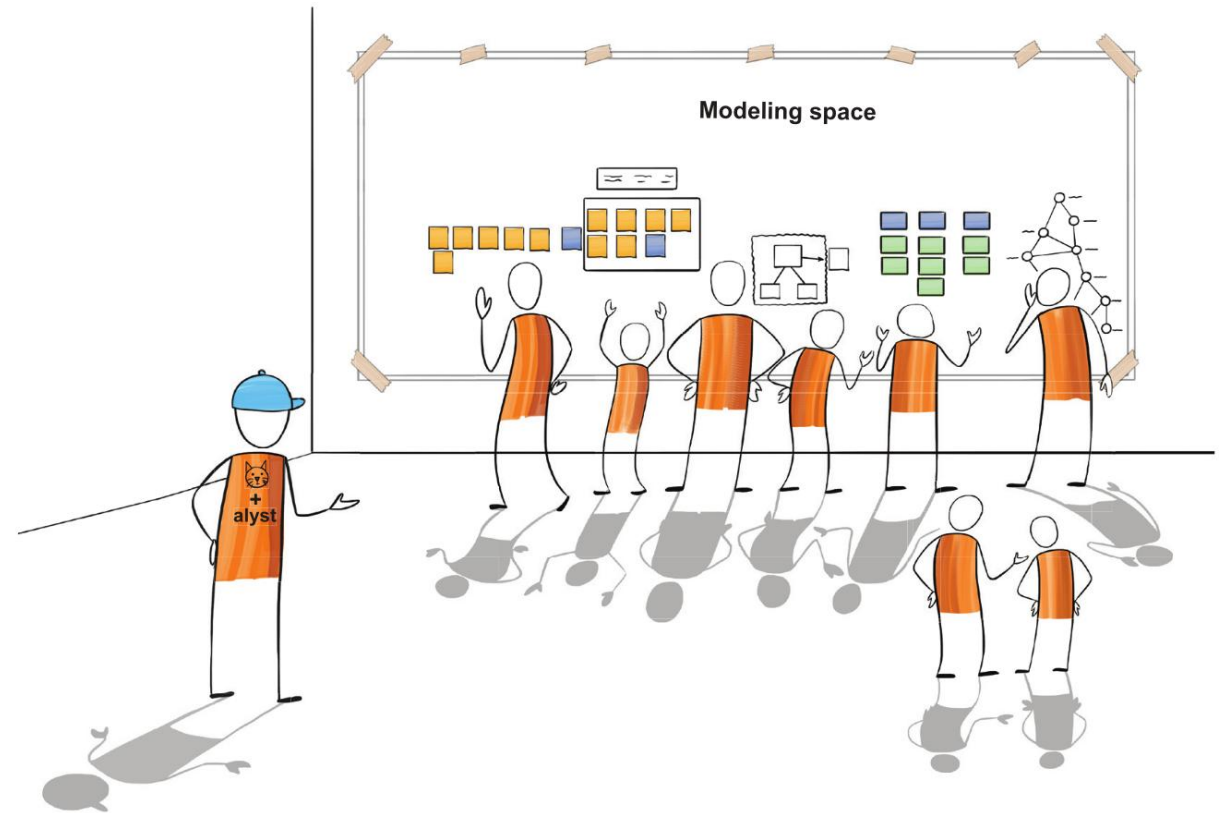


Image Reference: [Collaborative Software Design](#), Evelyn van Kelle, Gien Verschatse, Kenny Baas-Schwegler

Background

- <https://comocamp.org/#como>



What is Collaborative Modeling?

The essence of Collaborative Modeling lies in **cooperation** and **knowledge sharing**.

All participants bring in their different perspectives, expertise, and experiences.

This leads to a shared understanding of the problem, which helps to develop a shared software-related solution.

The goal is to achieve better and more innovative solutions through collaboration.

"Some Collaborative Modelling methods and mashups are:

- EventStorming
- User Story Mapping
- Impact Mapping
- Domain Storytelling
- Storystorming
- Context Mapping
- Example Mapping
- Business Model Canvas
- Bounded Context Canvas
- Scenario Casting
- Event Modeling
- Heuristics Mapping"

Common CoMo practices are listed on the CoMo Unconference website:

<https://comocamp.org/#como>



Summary: Collaborative Modelling is...

- Driven by industry, not academia
- Modern, agile touch on well established engineering practices
- Emphasis on **people and interactions** rather than automation and correctness
 - In line with values in [Agile Manifesto](#) (note that items on the right do have value!)
 - **Formal models** with UML, BPMN, etc. **are still valueable** for precision and towards implementation.
- Recommendation or „**best practice**“: **Start collaboratively**, then **refine** into more formal models as needed.

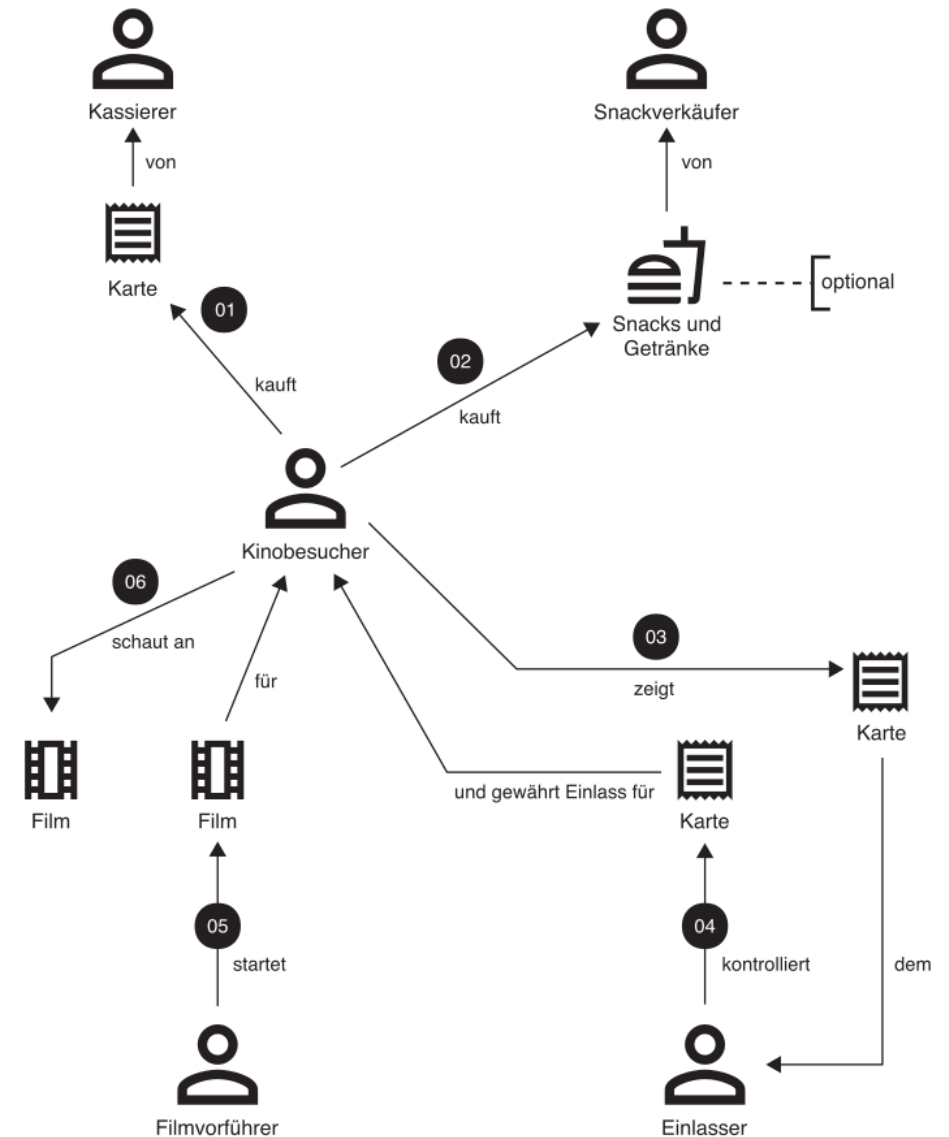
We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Domain Storytelling

- **Pictographic language** readable for domain experts (rich pictures)
 - Actors / “Akteure”
 - Work objects / “Arbeitsgegenstände”
 - Activities / “Aktivitäten”
 - Additionally: sequence number, notes, grouping
- domainstorytelling.org
- Collaborative; modeling canvas, general-purpose or specific tools
 - egon.io

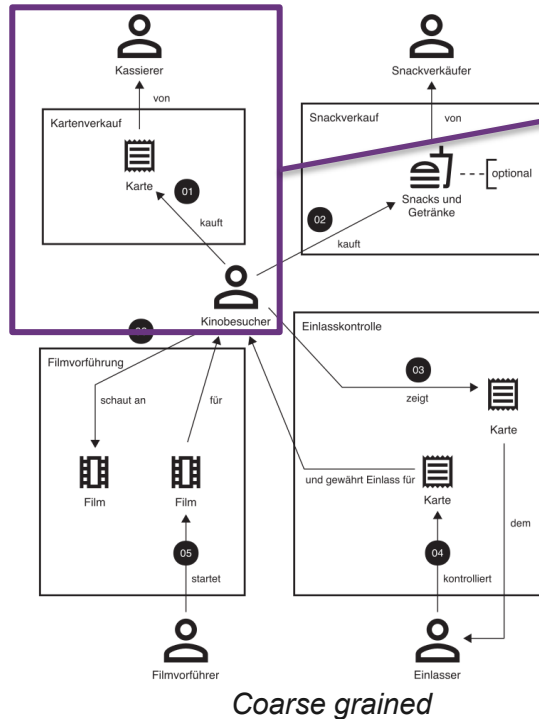


Example Source: [Domain Storytelling](http://domainstorytelling.org), Stefan Hofer, Henning Schwentner

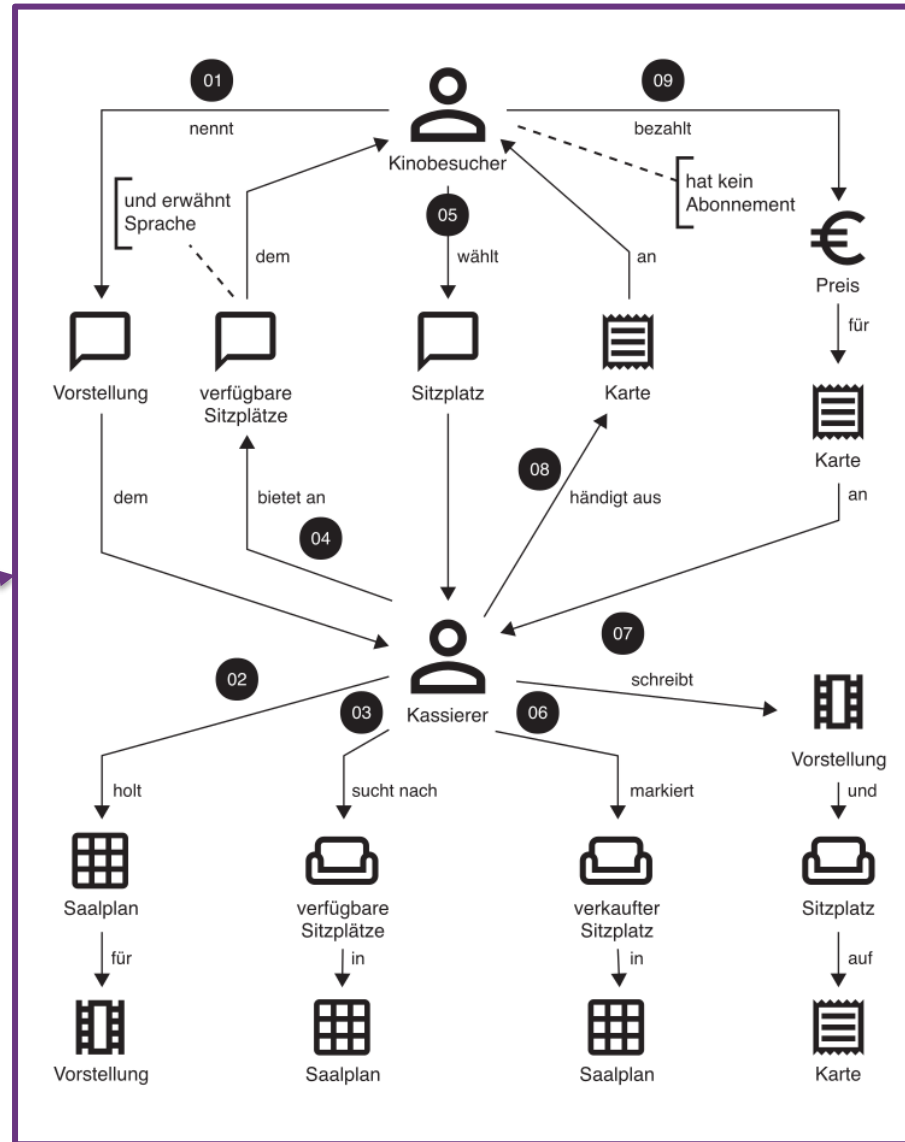
Domain Storytelling

Levels of Granularity

- **Coarse** grained («Grobgranular»)
- **Medium** grained («Mittelgranular»)
- **Fine** grained («Feingranular»)



Coarse grained



Fine grained

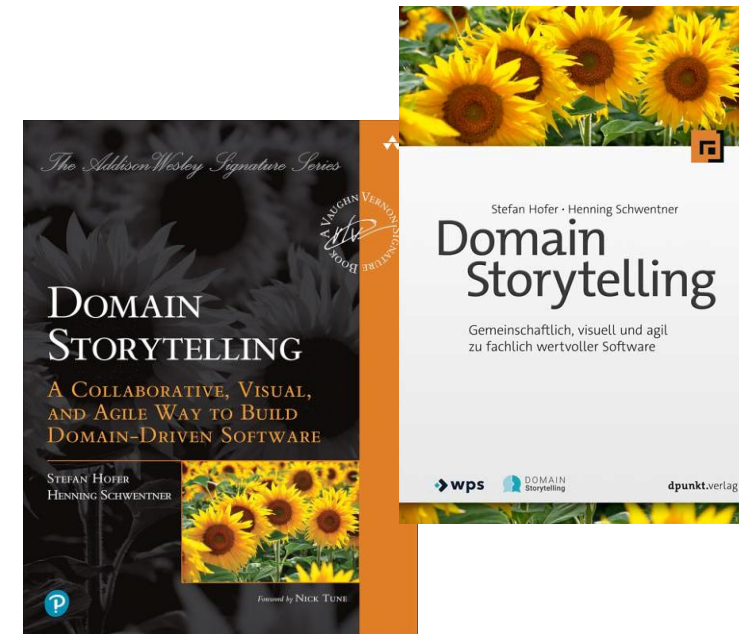
Images Source: [Domain Storytelling](#), Stefan Hofer, Henning Schwentner

Domain Storytelling

Book Recommendation

The book goes **deeper** into **use cases** and tells you how to use Domain Storytelling to:

- Learn the **domain language**
- Find **boundaries**
 - Strategic DDD
 - Bounded Contexts
- Derive **requirements** from domain stories
 - User stories
 - User story mapping
- Derive **domain model** from domain story
 - Implementing it in code
- and more ...



GERMAN: dpunkt.de/produkt/domain-storytelling

ENGLISH: amazon.de/gp/product/0137458916

Collaborative Modelling

EventStorming

- „The smartest approach to collaboration beyond silo boundaries“
- Flexible workshop format for collaborative exploration of complex business domains
- eventstorming.com

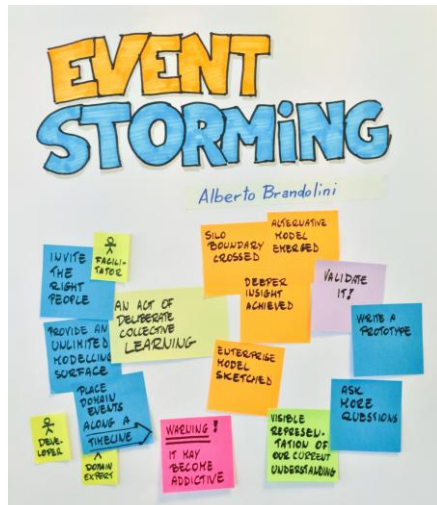
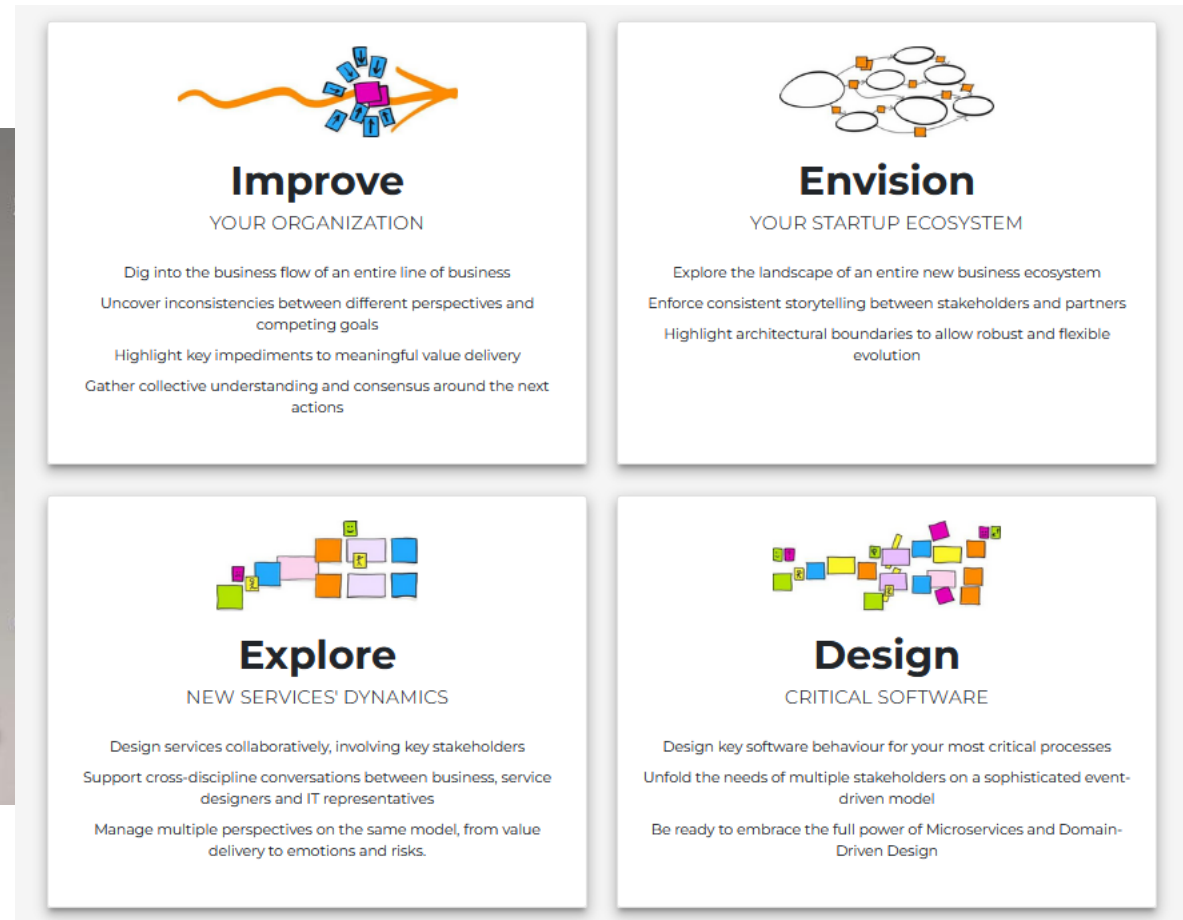


Image Source: Introducing EventStorming, Alberto Brandolini,
leanpub.com/introducing_eventstorming

Context and Motivation

- eventstorming.com



Domain Event: Essentials of Concept and Examples

- Something that **happened in the past**
(verb in past tense)
- Something the **business cares about**
(noun from ubiquitous language)
- On an **Orange** Sticky-Note
- More information:
 - Pattern definition in Evans/Vernon books
 - Explanation in Brandolini's LeanPub book
 - <https://www.innoq.com/en/blog/2019/01/domain-events-versus-event-sourcing/#domainevents>
 - <https://contextmapper.org/docs/event-storming/>



Case Study „Innovationsbox“

Let's do some practical storming ...



Innovationsbox: Product Vision

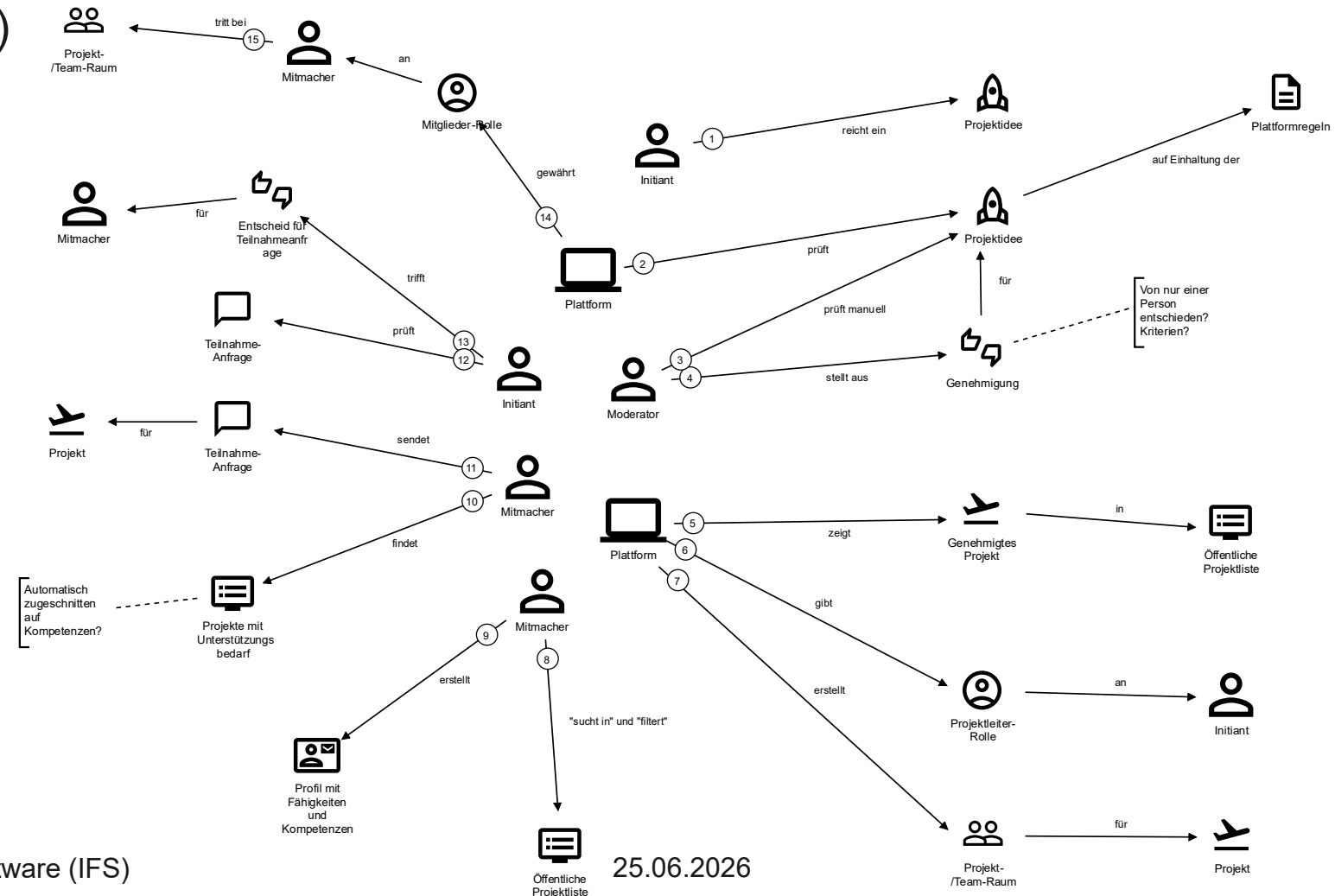
- „Für Menschen, Unternehmen und Organisationen, die klimafreundliche *Ideen* für das Glarnerland entwickeln möchten,
- ist die Glarner Innovationsbox eine *digitale Plattform*, auf der *Ideen sichtbar gemacht*, *Mitstreiter gefunden* und *Projekte gemeinsam weiterentwickelt* werden können.
- Im Unterschied zu klassischen Innovationswettbewerben steht nicht die Bewertung von Ideen im Vordergrund, sondern deren *gemeinschaftliche Weiterentwicklung und Vernetzung*.“

Case Study

Innovationsbox: Grobgranulare Domain Story

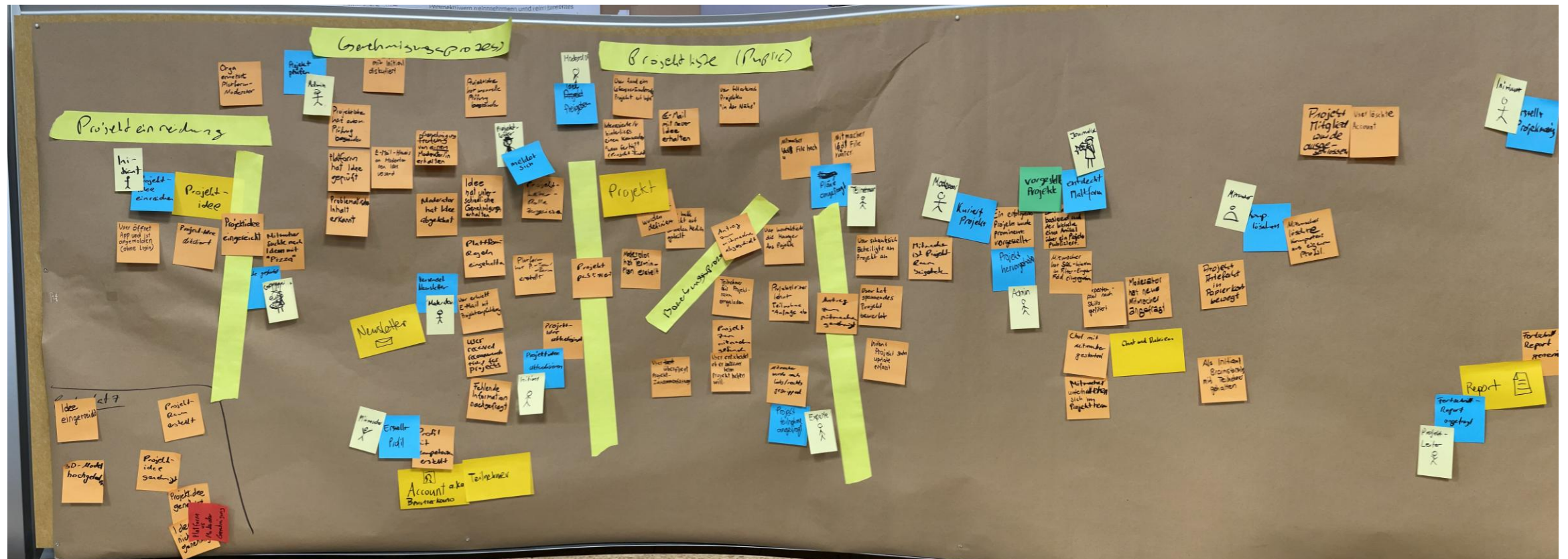
- (so wie ich es verstanden habe :)

Innovationsbox



Case Study

Lets go ...



EventStorming

Some more details ...

Image Source: Introducing EventStorming, Alberto Brandolini, leanpub.com/introducing_eventstorming



EventStorming

Three Types of EventStorming's

- **Big Picture**
 - Create a shared state of mind
 - Explore business/domain model
 - Identify Bounded Contexts
- **Process Modelling**
 - Assess specific process
 - Find bottlenecks and identify parts of the system to decouple from the existing software
- **Software Design**
 - Design clean and maintainable Event-Driven software
 - Derive shared language and (domain) model within a Bounded Context

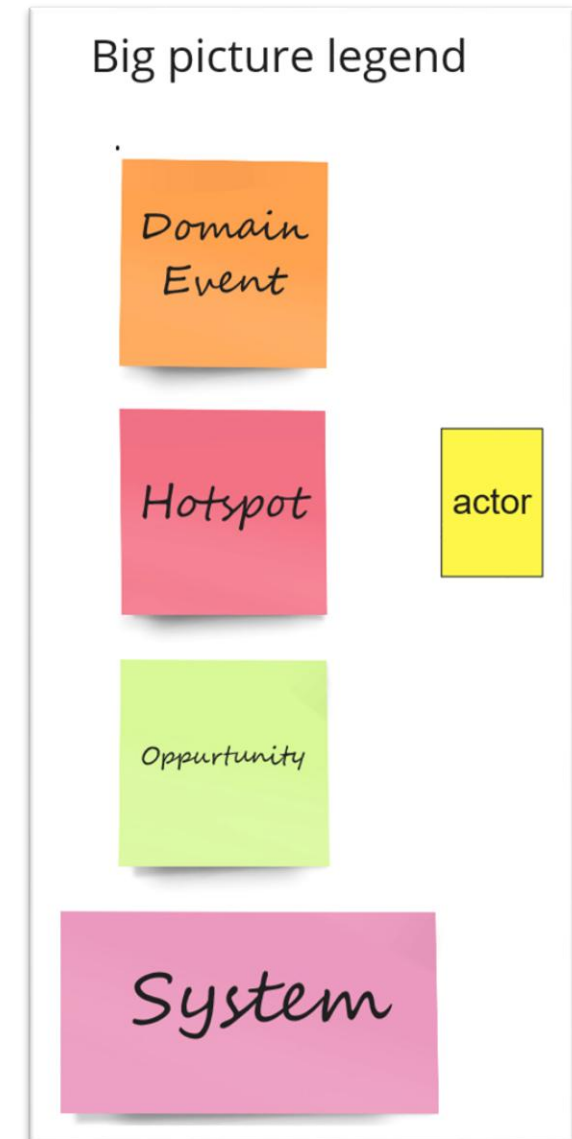


Image Source: DDD Crew, EventStorming Cheatsheet, github.com/ddd-crew/eventstorming-glossary-cheat-sheet

Concepts Overview for Process Modelling

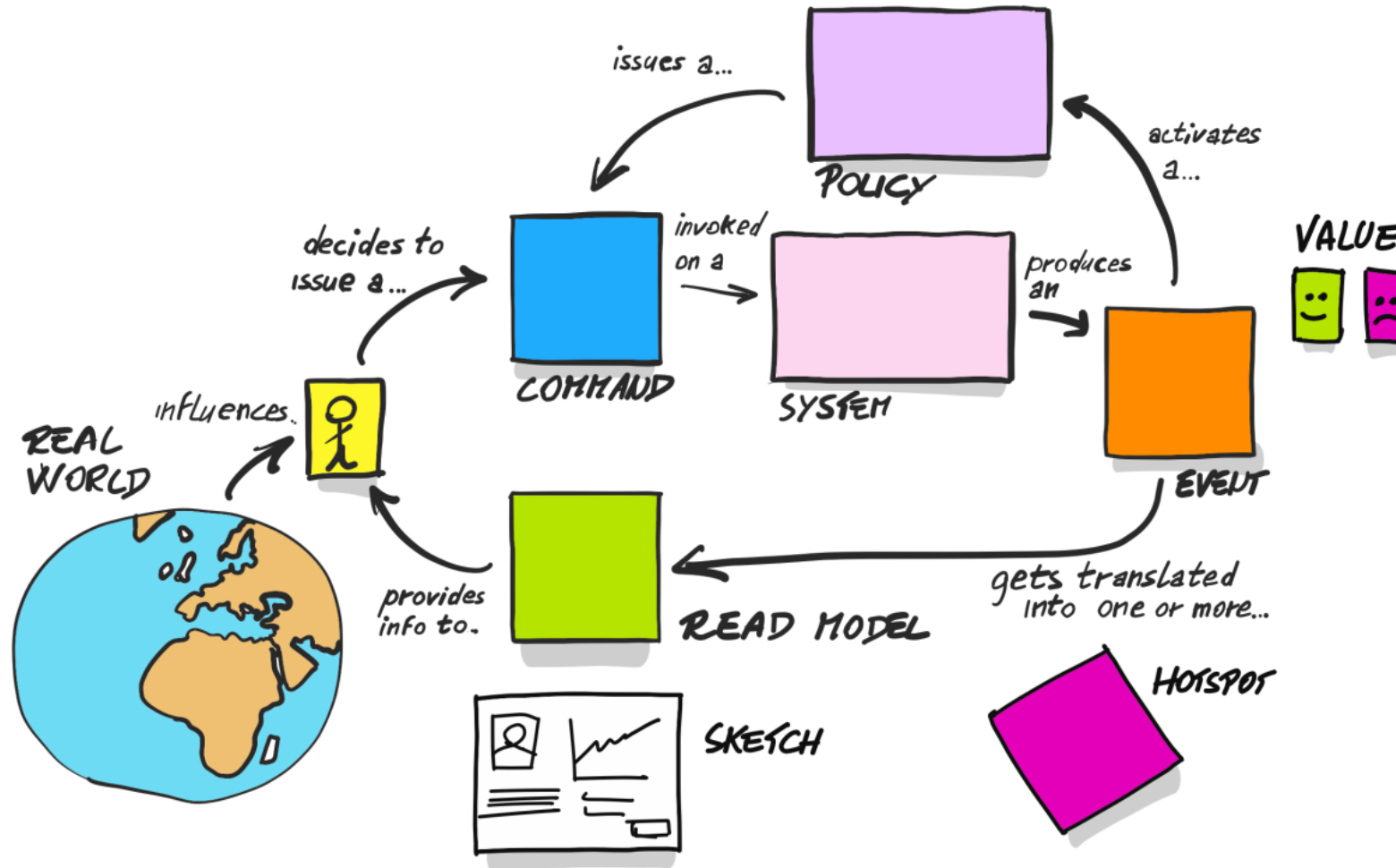


Image Source: Introducing EventStorming, Alberto Brandolini, leanpub.com/introducing_eventstorming

Design-Level EventStorming

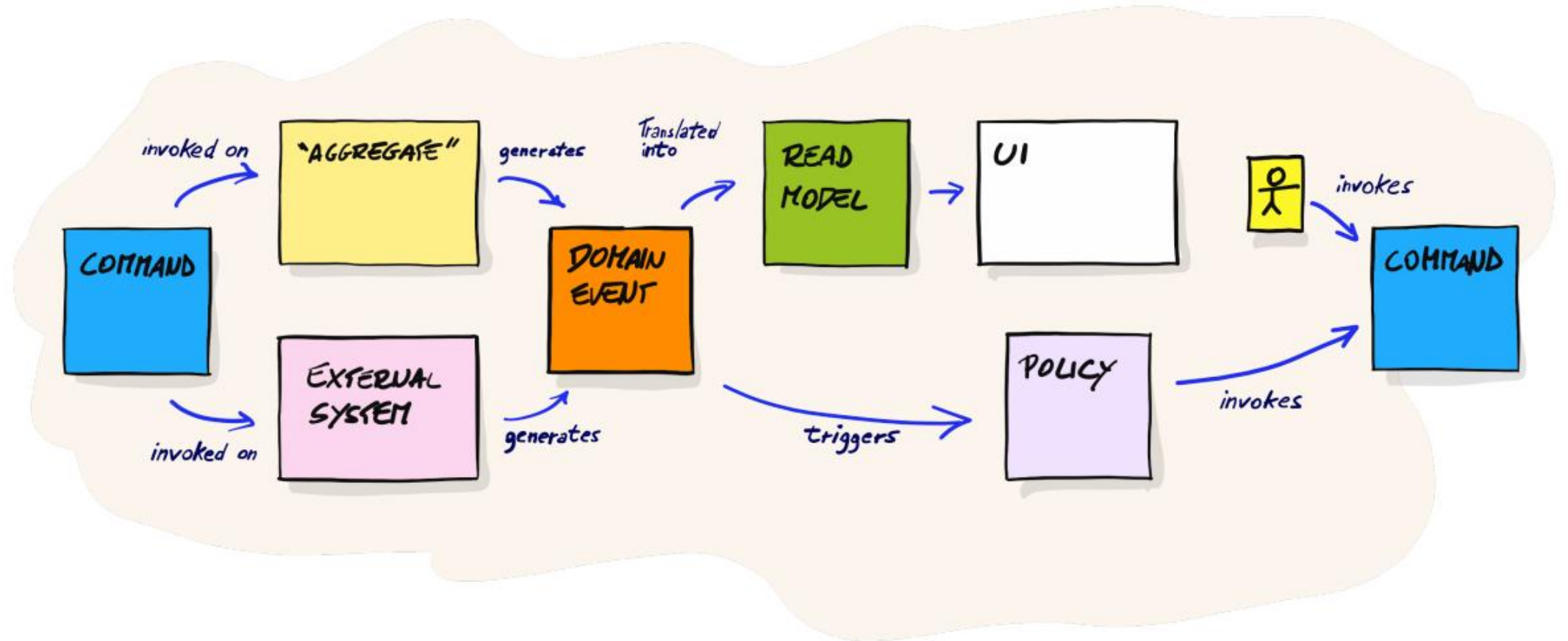


Image Source: Introducing EventStorming, Alberto Brandolini, leanpub.com/introducing_eventstorming

EventStorming

But: There are different «schools» ...

- ... and interpretations.
- DDD crew design-level notation:

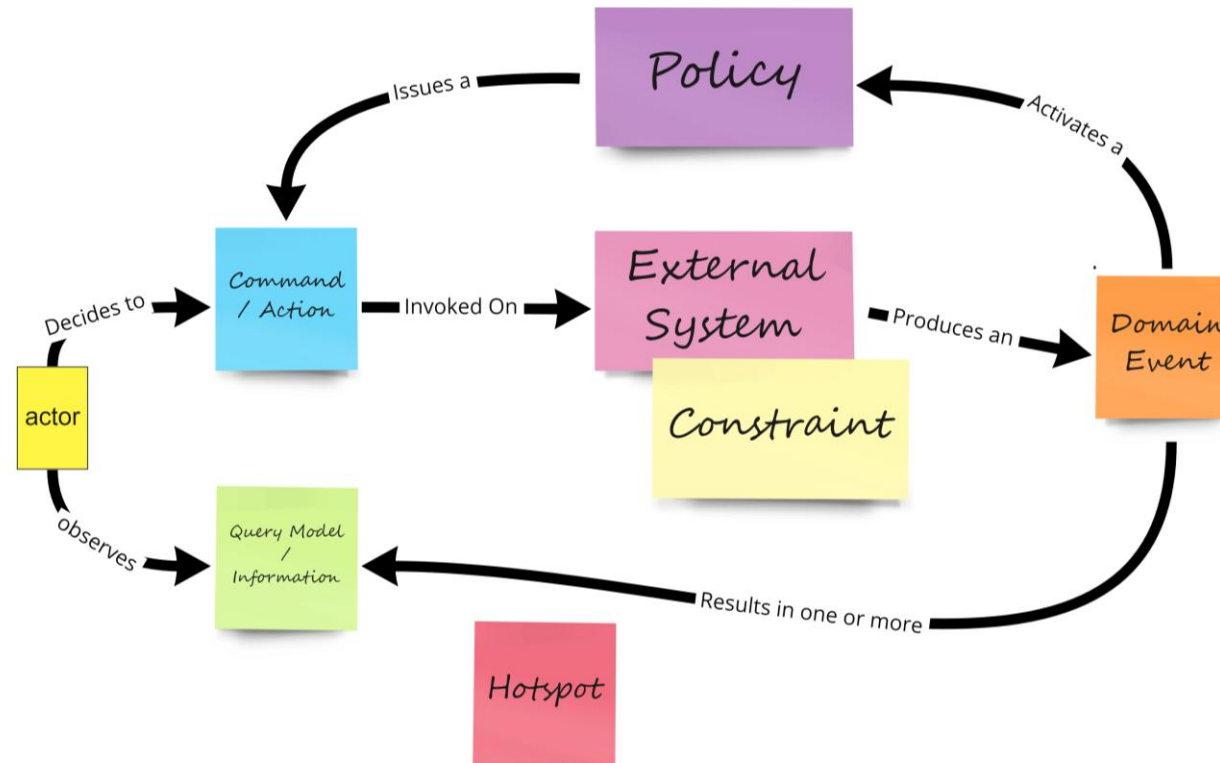


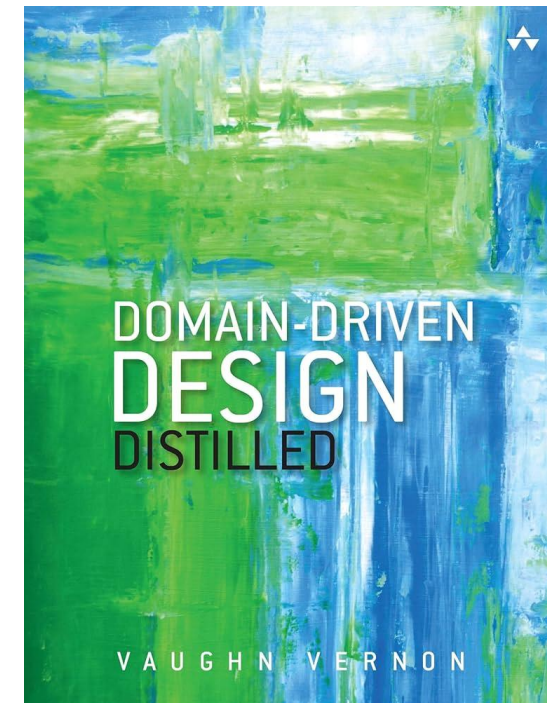
Image Source: DDD Crew, EventStorming Cheatsheet, github.com/ddd-crew/eventstorming-glossary-cheat-sheet

Recommendation

- Clarify the **purpose**
- Start **simple**
- Match scale to **scope**
 - Big Picture for broad organizational storytelling, Process/Design level for detailed workflows, and Software Design for implementation conversations.
- **Experiment** and iterate
- Lean on **facilitation**, not just technique
- Stay **pragmatic**
- **Blend** styles if needed

For our case study we followed the steps recommended by Vaughn Vernon in his «DDD Distilled» book:

amazon.com/Domain-Driven-Design-Distilled-Vaughn-Vernon/dp/0134434420



„Bridge“ to Strategic Domain-Driven Design (DDD)

- CoMo practices can be useful tools to **decompose** your domain into **Bounded Contexts**!

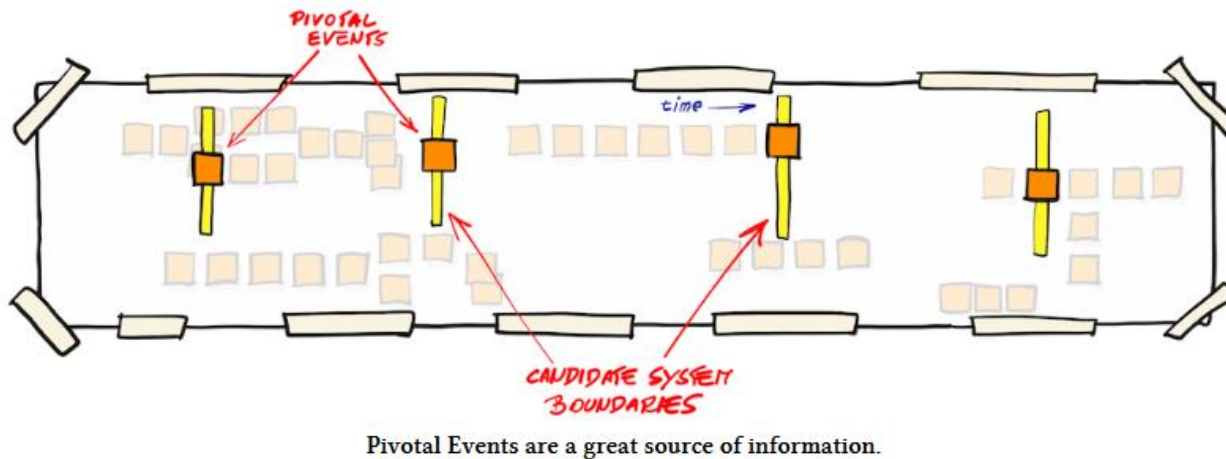


Image Source: Introducing EventStorming, Alberto Brandolini, leanpub.com/introducing_eventstorming

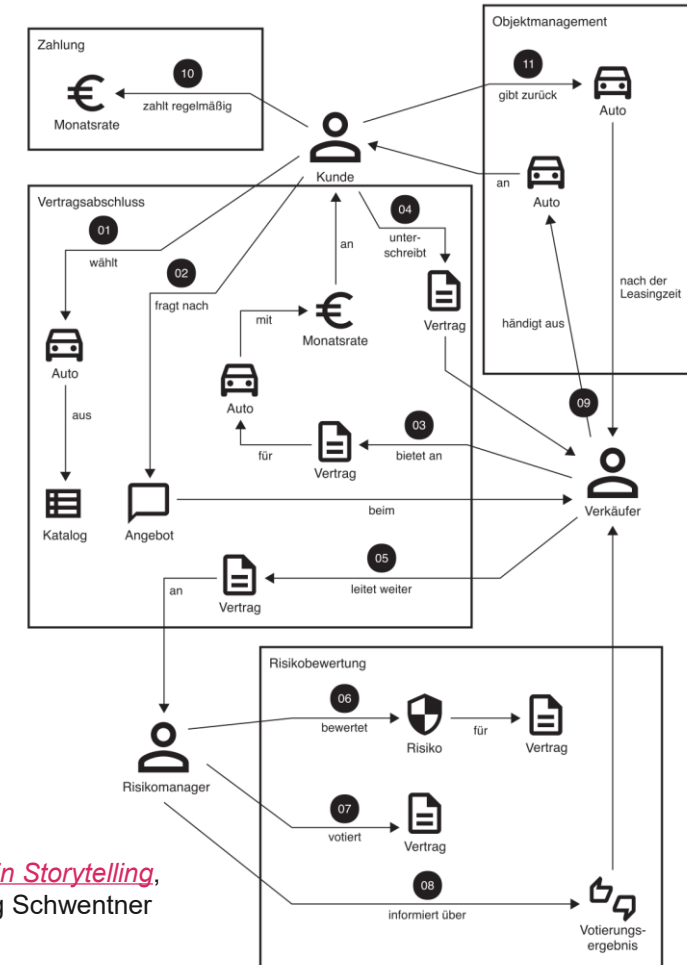
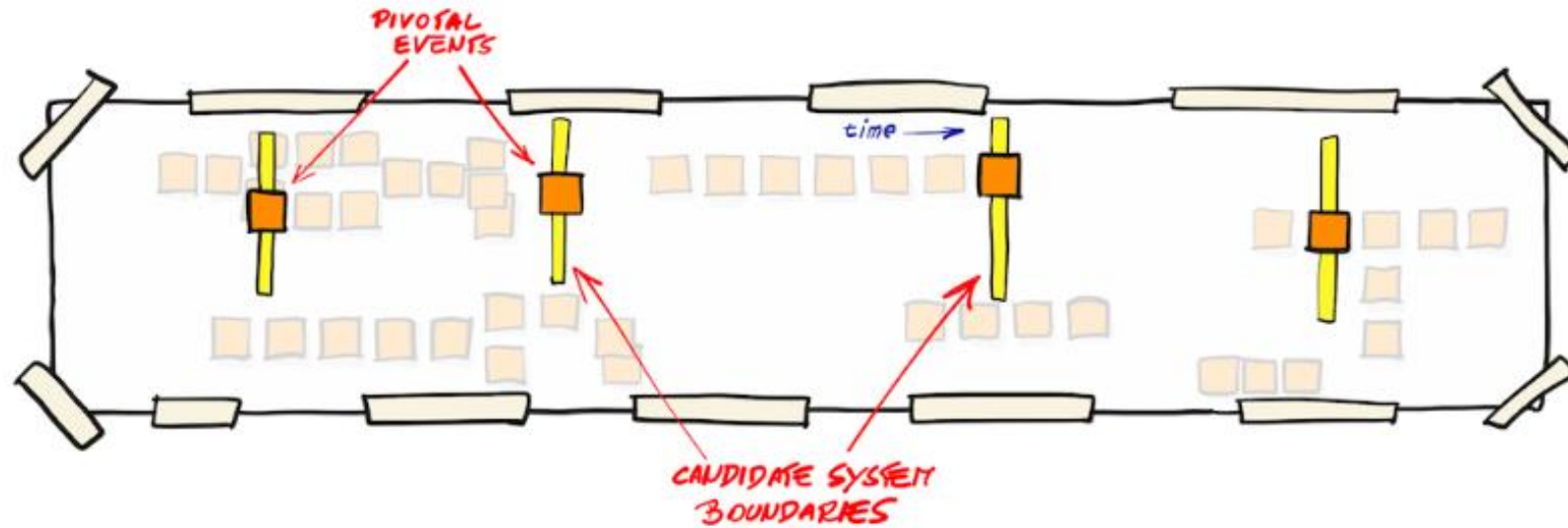


Image Source: *Domain Storytelling*,
Stefan Hofer, Henning Schwentner

Pivotal Events

- One important concept to decompose your domain, are **pivotal events**:
 - Or how Brandolini puts it: «the expert on the left and the ignorant on the right»



Pivotal Events are a great source of information.

- How to identify those? 🤔

Image Source: Introducing EventStorming, Alberto Brandolini, leanpub.com/introducing_eventstorming

Heuristics for the Identification of Pivotal Events (by M. Plöd)

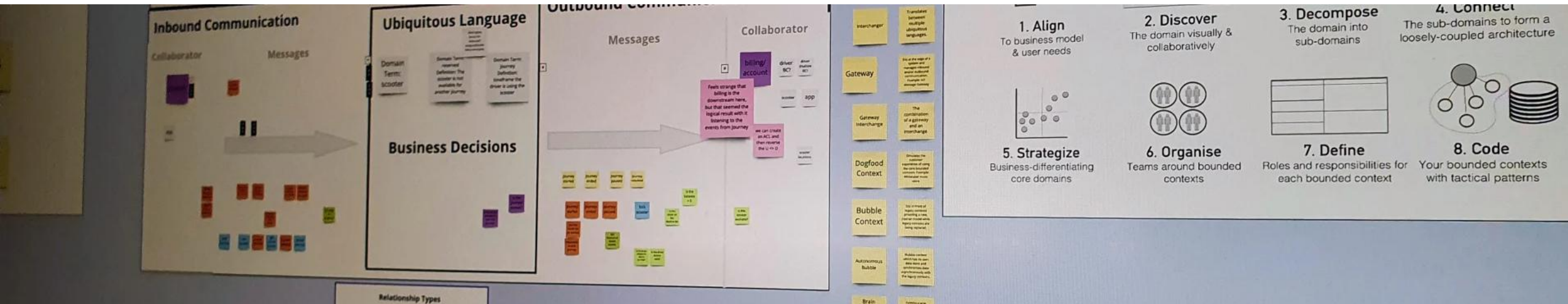
1. **Events That Trigger Significant Downstream Activity:** The "So What?" test.
2. **Events That Represent Key Business Decisions or Policy Enforcement:** Decision points and policy application.
3. **Events That Involve Hand-Overs to External Parties:** Boundary crossings and external triggers.
4. **Events That Lead to Lasting State Changes:** Significant data updates and process milestones.
5. **Events That Indicate Potential Bottlenecks or Failure Points:** Points of contention and exception handling.
6. **Compliance and Regulatory Significance:** Events tied to legal or regulatory requirements.
7. **Change in Resource Allocation:** Events causing significant shifts in personnel, time, or budget.
8. **Impact on Multiple Stakeholders:** Events affecting various departments or stakeholders.
9. **High Business Value Realization Potential:** Events that correlate with achieving key business objectives.

Source: Finding the Turning Points: Pivotal Events in Big Picture EventStorming, Michael Plöd, michael-ploed.com/blog/finding-the-turning-points-pivotal-events-in-big-picture-eventstorming

Strategic Domain-Driven Design (DDD)

An Approach for Service Decomposition

Image Source: Strategic DDD using the Bounded Context Canvas, ddd.academy/strategic-ddd-using-bounded-context-canvas-nick-tune



Strategic vs. Tactical Domain-driven Design

- **Strategic DDD**

- Bounded Contexts
- Relationships between Bounded Contexts (Context Mapping)
- How do Bounded Contexts integrate

- **Tactical DDD**

- Design of domain model inside a Bounded Context
- Patterns such as: Aggregate, Entities, Value Objects, Domain Events, etc.

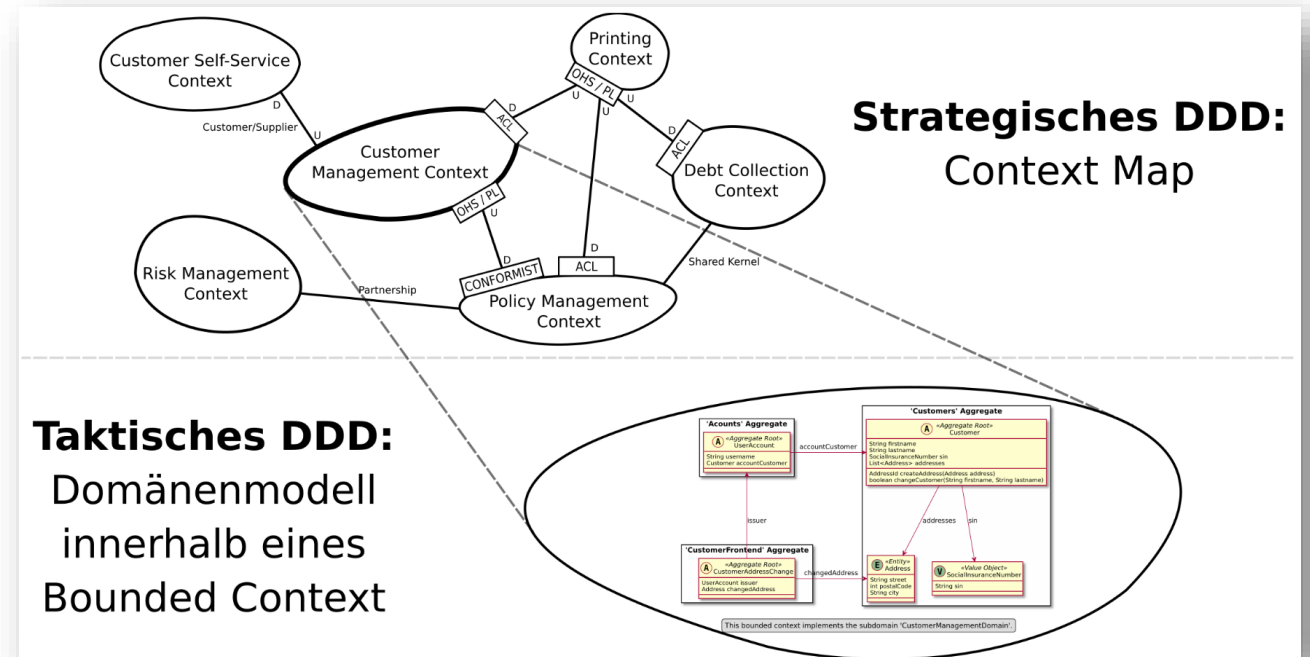
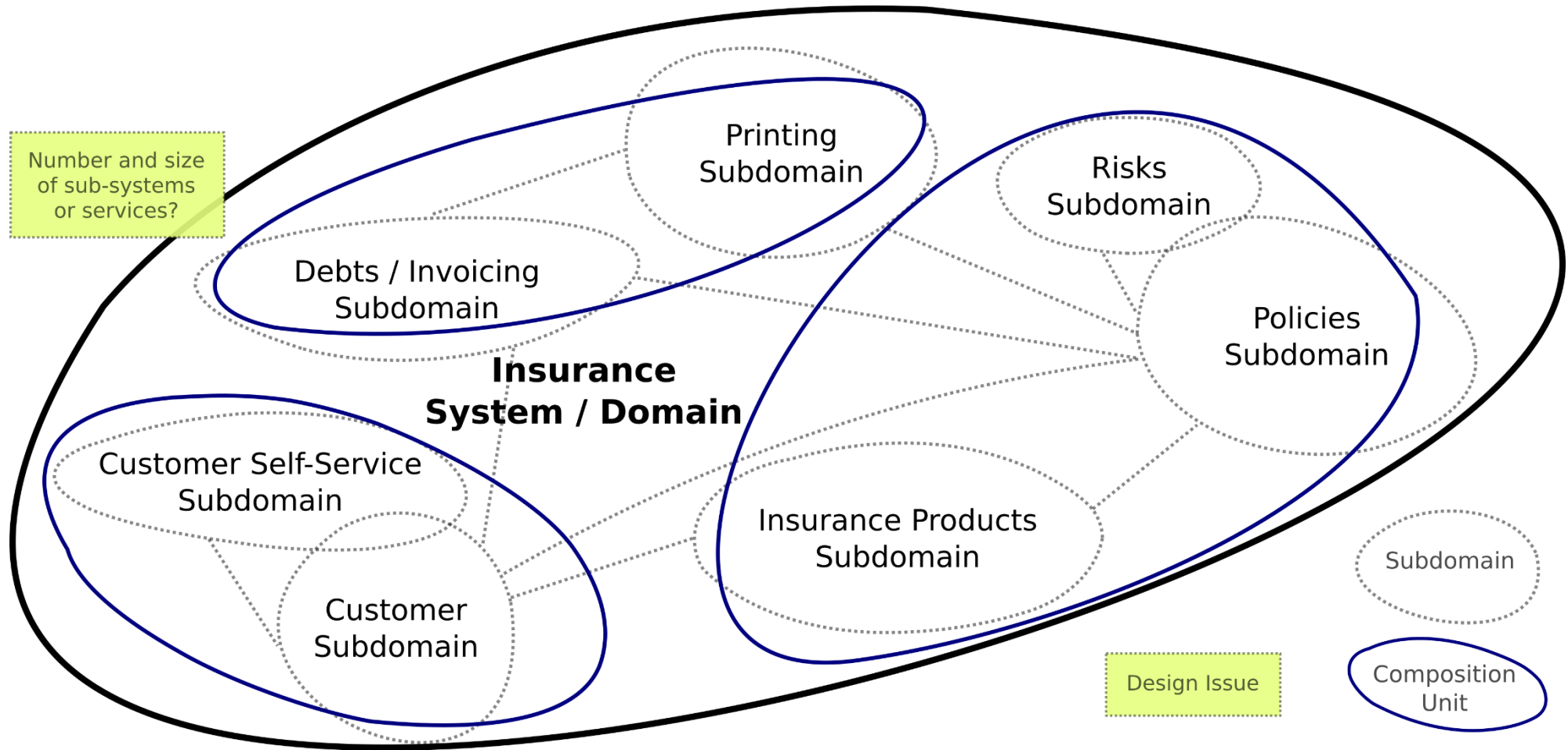


Image Source: Stefan Kapferer & Olaf Zimmermann, «Domain-Driven Design in der Praxis - Erfahrungen mit dem Open-Source-Tool Context Mapper», [JavaSPEKTRUM Artikel](#)

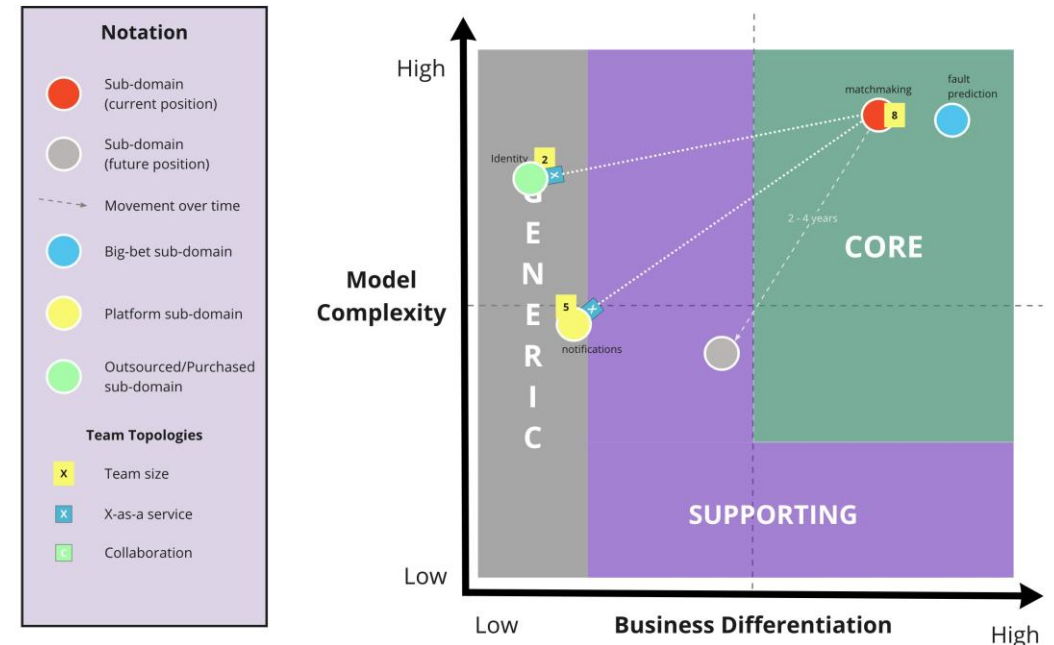
How to decompose a system into services/modules?



Strategic Domain-Driven Design (DDD)

Subdomains

- **Three types** of subdomains:
 - **Core domain:** These subdomains represent the core concepts of your business domain (most important parts).
 - **Supporting domain:** Parts that support your business but do not belong to your core competences. They are vital for your core domain to run; but have no strategic relevance.
 - **Generic subdomain:** Parts that are needed but do not capture or communicate core business knowledge. Not critical.



Source: Core Domain Charts by DDD Crew: github.com/ddd-crew/core-domain-charts

Bounded Context

- DDD suggests to **decompose system into Bounded Contexts**.
- Bounded Contexts establish boundaries between **domain models**.
- Words/concepts must have a **precise/distinctive meaning** inside a Bounded Context.
 - Establishes the so-called "ubiquitous language"¹.
- Therefore, Bounded Contexts establish **linguistic boundaries**.
- Can also be seen as abstractions of **(sub-)systems** and development **teams**.
- A Bounded Context realizes **parts of** one or multiple **subdomains**

¹Reference: Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley (2003)

Linguistic Boundaries

Words can have different meanings in different contexts!

Web Application Context

"Account" =
User account to
login ...

Banking Context

"Account" =
Banking account
...

Illustration Reference: Alberto Brandolini, *Strategic Domain-Driven Design with Context Mapping*, InfoQ Article: <https://www.infoq.com/articles/ddd-contextmapping/>

Subdomains vs. Bounded Contexts

- Bounded Contexts implement parts of one or multiple subdomains.
- Subdomains: **problem space** (result of **Object-oriented analysis – OOA**)
- Bounded Contexts: **solution space** (result of **Object-oriented design – OOD**)

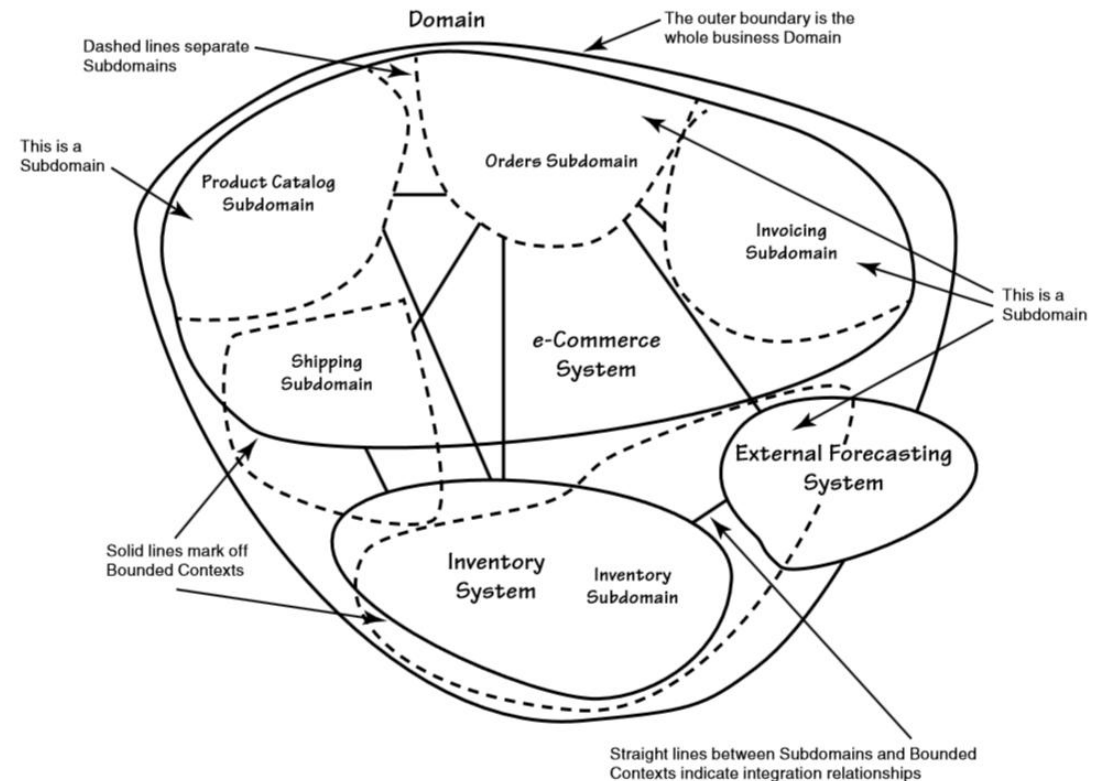
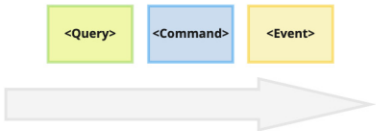


Image Reference: Vaughn Vernon, Implementing Domain-driven Design, [Link](#)

Strategic Domain-Driven Design (DDD)

Bounded Context Canvas

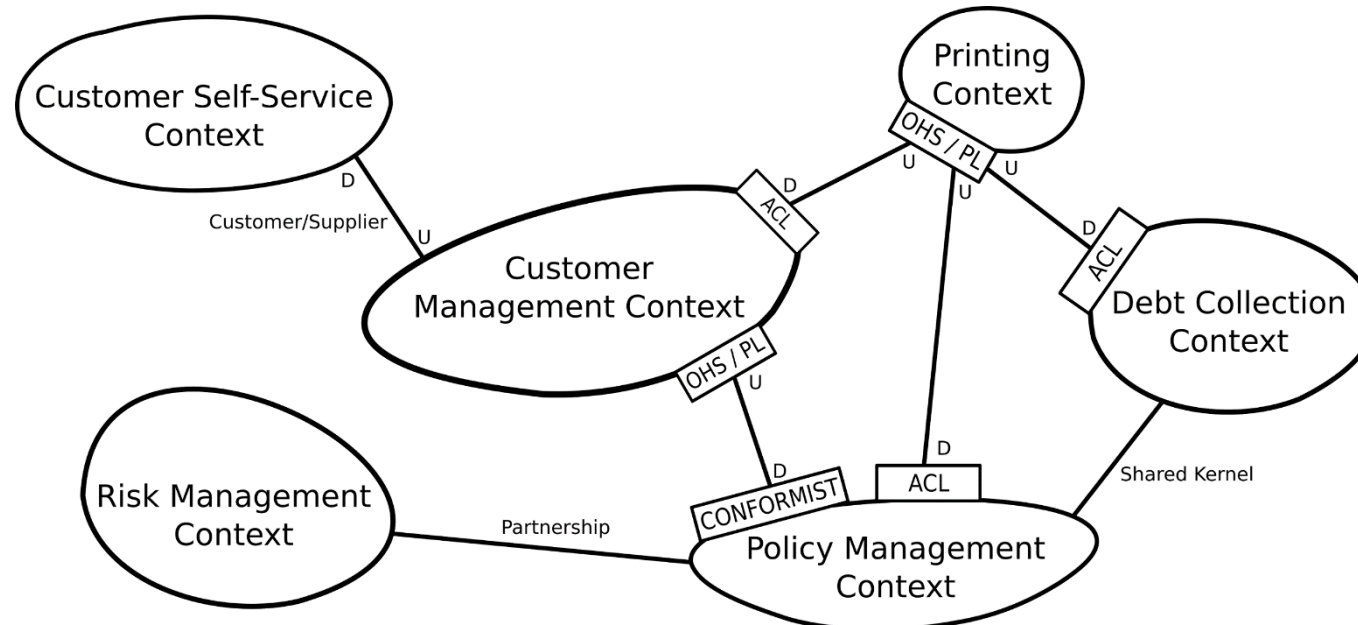
Name:		V5 github.com/ddd-crew/bounded-context-canvas					
Purpose What benefits does this context provide, and how does it provide them? Describe the purpose from a business perspective	Strategic Classification <table border="0"><tr><td>Domain - core - supporting - generic - other? </td><td>Business Model - revenue - engagement - compliance - cost reduction </td><td>Evolution - genesis - custom built - product - commodity </td></tr></table>			Domain - core - supporting - generic - other?	Business Model - revenue - engagement - compliance - cost reduction	Evolution - genesis - custom built - product - commodity	Domain Roles Role Types - draft context - execution context - analysis context - gateway context - other
Domain - core - supporting - generic - other?	Business Model - revenue - engagement - compliance - cost reduction	Evolution - genesis - custom built - product - commodity					
Inbound Communication Collaborator Messages  Ubiquitous Language Context-specific domain terminology  Business Decisions Key business rules, policies, and decisions  Outbound Communication Messages Collaborator 							
Assumptions Describe which currently unverified assumptions went into this bounded context design. Make those assumptions explicit by documenting them here	Verification Metrics Describe metrics which can be used to (in)validate the current structure of this bounded context?		Open Questions				

Source: Bounded Context Canvas by DDD Crew:
github.com/ddd-crew/bounded-context-canvas

Strategic Domain-Driven Design (DDD)

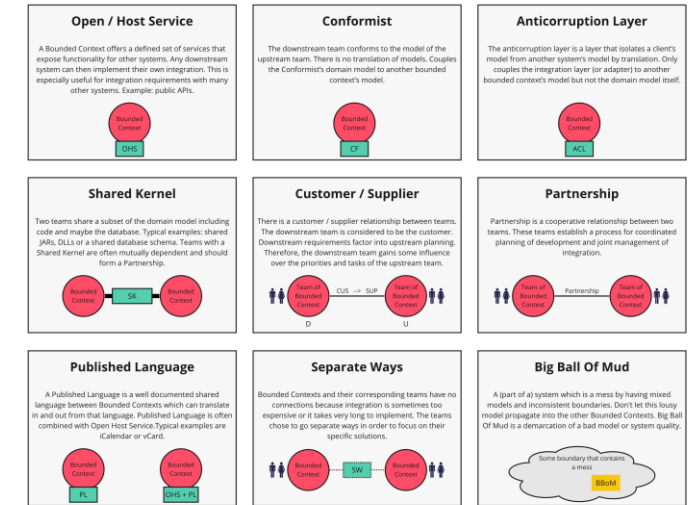
Context Mapping

- Define how **Bounded Contexts** integrate.
- Illustrate "Information flow" (from "Upstream" to "Downstream")



Context Map Cheat Sheet

Context Map Patterns



Team Relationships



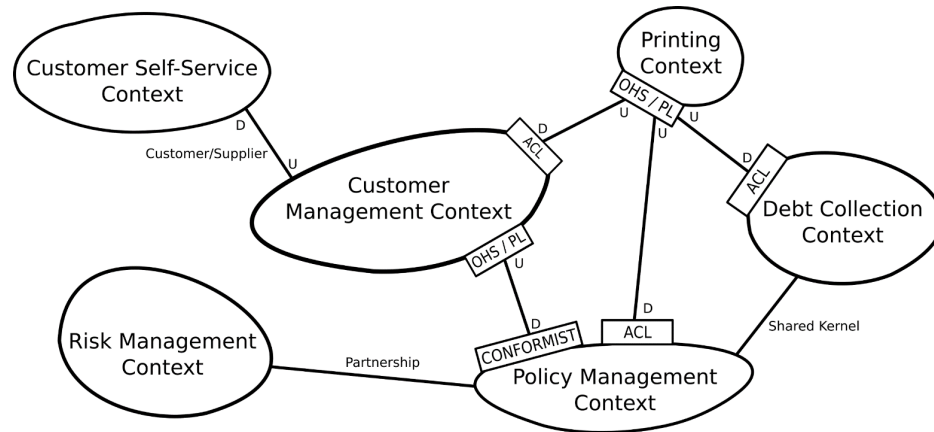
Context Map Cheat Sheet v0.1 | <https://github.com/ddd-crew/context-mapping> | License: Creative Commons Attribution-ShareAlike v4.0 | Contains quotes from the DDD Reference by Eric Evans | <https://martinfowler.com/ddd/ddd.html> | Reference: 2013-2024

github.com/ddd-crew/context-mapping

Legend: Upstream (U), Downstream (D), Open Host Service (OHS), Published Language (PL), Anticorruption-Layer (ACL)

Upstream-Downstream and Customer-Supplier

- Upstream-Downstream relationships are **asymmetric** relationships.
- Information **flow from upstream [U] to downstream [D]**
 - "flow" metaphor
- Customer-Supplier: **Special kind of upstream-downstream** relationship
- Customer (downstream) has high impact/power on features realized by supplier (upstream)
- Supplier respects customers requirements and plans development accordingly.



Partnership

- **Symmetric relationship** (no clear upstream/downstream)
- A **cooperative relationship** in which two teams can only succeed or fail together.
 - Delivery failure of one leads to delivery failure of both.
- Such teams may share CI/CD infrastructure and **typically release together**.

Shared Kernel

- **Symmetric relationship** (no clear upstream/downstream)
- Two Bounded Contexts **share a part of their domain models**
- Shared part typically realized as **library maintained by both teams**

Open Host Service (OHS)

- **Upstream** context which provides **uniform API** to multiple downstreams
- If you have **multiple downstreams** with mostly **same requirements** implement one **single API** instead of integrating with all individually

Published Language (PL)

- **Upstream** context defines a **common language** that is used to **translate** between the models (upstream vs. downstream)
 - **Well-documented shared language (common medium)** for communication
 - Often combined with OHS.
-
- “Widely known examples of a Published Language are iCalendar or vCard.” ([DDD Crew](#))

Anticorruption-Layer (ACL)

- **Downstream** context **translates between domain models**
- Upstream model is different and downstream wants to **protect** its own model from changes of the upstream
- **Isolation-/Translating-Layer**
 - Provide upstreams features in terms of the own domain model
- Opposite solution to "Conformist"
 - Only ACL or CF makes sense, but not both

Conformist (CF)

- **Downstream** decides to simply **conform to the upstreams domain model**
 - Changes to the upstream model have impact on downstream
 - In this case downstream **does not need an ACL**
-
- Opposite solution to ACL
 - Only ACL or CF makes sense, but not both

Summary & Outlook

What's next?



Summary & Outlook

Summary: „My Interpretation“ (Context Mapper Meta-Model)

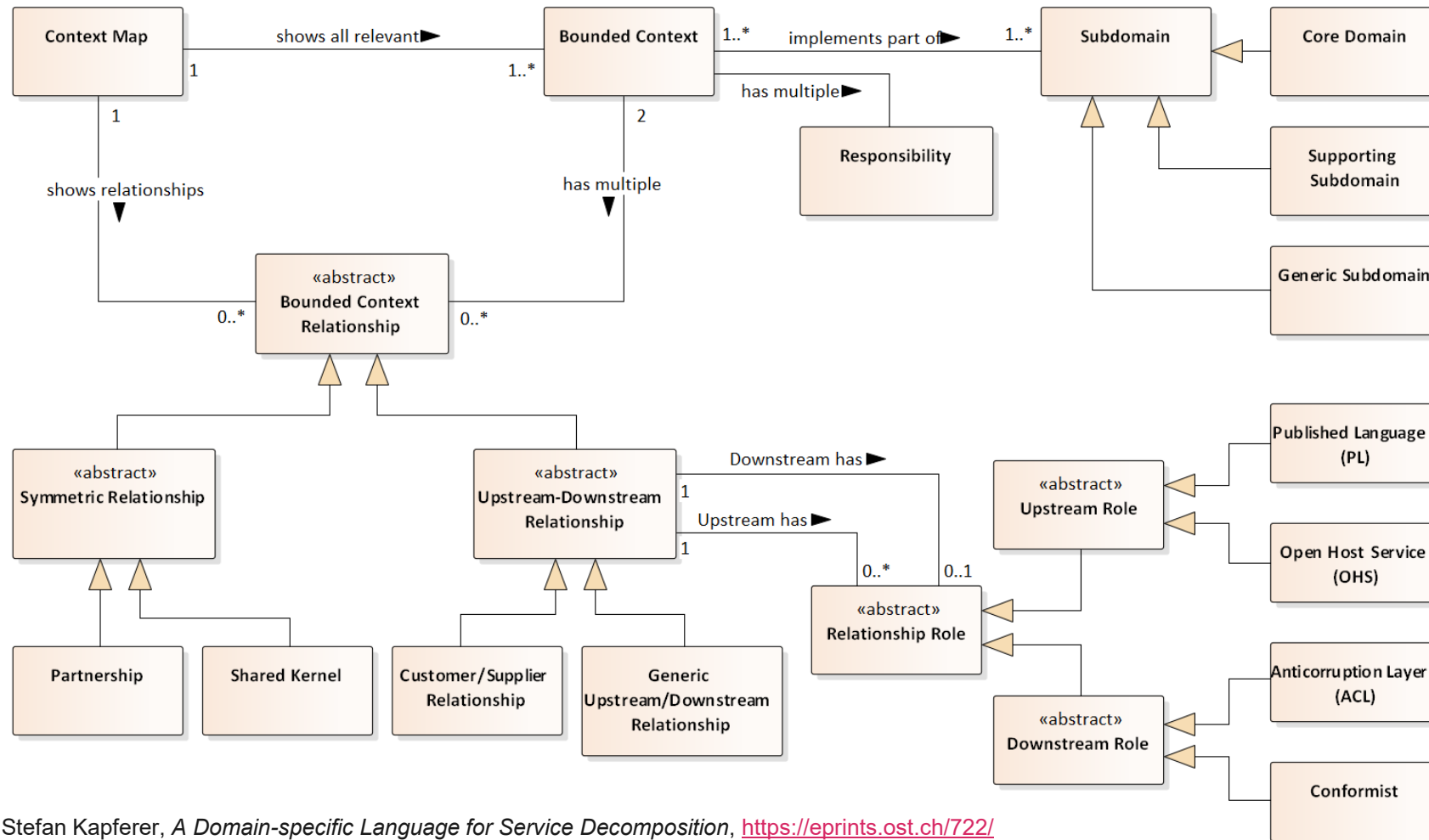


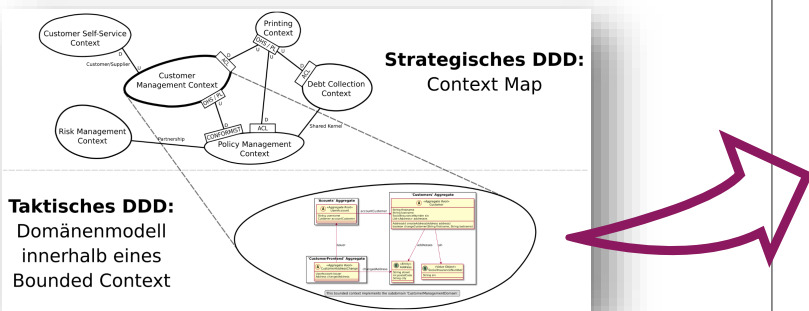
Image Reference: Stefan Kapferer, *A Domain-specific Language for Service Decomposition*, <https://eprints.ost.ch/722/>

On the connection between CoMo and DDD ...

- All practices shown and applied today can help to discover subdomains and Bounded Contexts (**Strategic DDD**)
 - With EventStorming use pivotal events as discussed
 - With Domain Storytelling use „grouping“ notation
- Both, EventStorming and Domain Storytelling, generate output for a domain/data model ... and therefore help for **Tactic DDD** later
 - With EventStorming use „data“ („Aggregate“) stickies
 - With Domain Storytelling look at the modelled „work objects“
- Both, DST and ES, establish „**Ubiquitous Language**“

Summary & Outlook

Therefore, next step:

- **Model domain** for each Bounded Context
 - Maybe conduct another **design-level ES** per context
 - Use **tactic DDD patterns** to model the domain
- 
- Tactic DDD for one of our contexts could be another meetup session, if you are interested ;)
 - There we could even write some code ☺

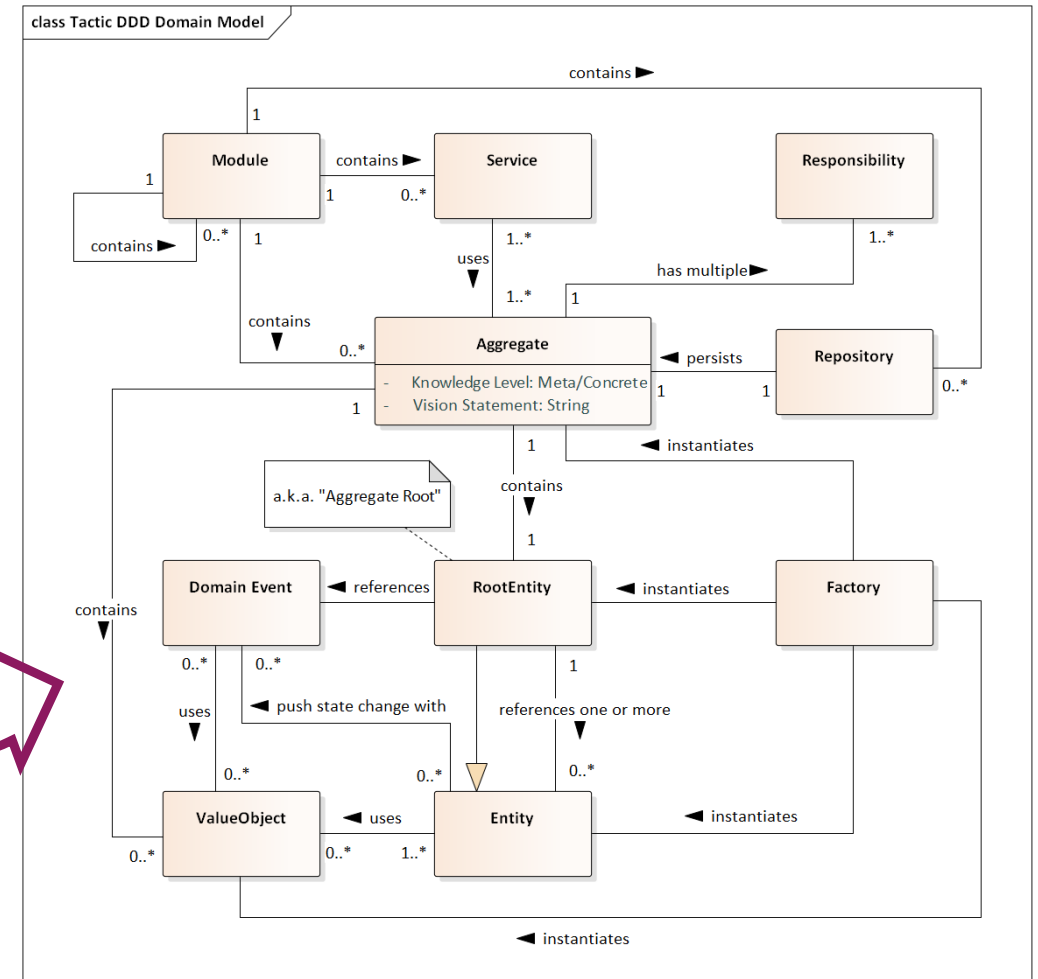


Image Reference: Stefan Kapferer, *A Domain-specific Language for Service Decomposition*, <https://eprints.hsr.ch/722/>

Questions & Discussion

Thanks for attending the meetup!

